Assignment No	**14**
Name	**Satish Rathod**
Title	**List Queue & Priority Queue**
Objective	**1) List implementation of Ordinary Queues** **2) Node Template and Enqueue function in Priority Queues**
Roll No	**MCA2649**

1) List Implementation of an Ordinary Queue

Source Code:-

```
//Name:Satish Rathod

//Roll No:MCA2649

//Module 4: Linked Lists

// Application of Linked Lists - List Implementation of an Ordinary Queue


import java.util.*;


//QNode Template

class QNode

{

        int data;

        QNode next;
```

```
        public QNode(int d)
        {
                this.data = d;
                this.next = null;
        }
}//end of the QNode

//Queue List
class LQueue
{
        QNode front;
        QNode rear;

        public LQueue()
        {
                front = null;
                rear = null;
        }
        //Enqueue
        public void Enqueue(int d)
        {
```

```
        QNode x = new QNode(d);


        if(front == null) // First Node in the Queue
        {
                front = x;
                rear = x;
        }
        else // Add x to the end of the Queue
        {
                rear.next = x;
                rear = x;
        }

}//end of Enqueue


//Display
public void Display()
{
        if(front == null)
        {
                System.out.println("\nQueue UnderFlow!");
        }
```

```
        else
        {
                QNode temp = front;
                System.out.print("\nQueue Contains: ");

                while(temp!=null)
                {
                        System.out.print(temp.data + " ");
                        temp = temp.next;
                }
        }
}//end of the display

// Peek Front
public void PeekFront()
{
        if(front == null)
        {
                System.out.println("Queue UnderFlow!");
        }
        else
        {
```

```
                System.out.print("\n\nElement At the Front: " +
front.data);
            }
        }//end of the PeekFront


        // Peek Rear
        public void PeekRear()
        {
            if(front == null)
            {
                System.out.println("Queue UnderFlow!");
            }
            else
            {
                System.out.println("\nElement At the Rear: " + rear.data);
            }
        }//end of the PeekRear


        //Dequeue
        public void Dequeue()
        {
            if(front == null)
            {
```

```
            System.out.println("Queue UnderFlow!");
        }
        else
        {
            int t = front.data;

            if(front == rear)
            {
                front =null;
                rear = null;
            }
            else
            {
                front = front.next;
            }
            System.out.print("\nElement removed: "+t);

        }
    }//end of Dequeue
```

```
}//end of the LQueue


class QList
{
	public static void main(String args[])
	{
		LQueue q = new LQueue();


		for(int i=1; i<=5; i++)
		{
			q.Enqueue(i*10);
			q.Display();
		}


		q.PeekFront();
		q.PeekRear();


		for(int i=1; i<=5; i++)
		{
			q.Dequeue();
			q.Display();
```

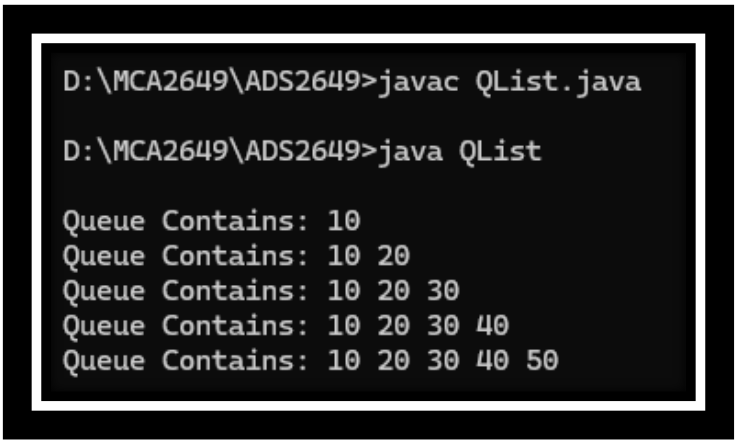
```
        }

    }//end of main

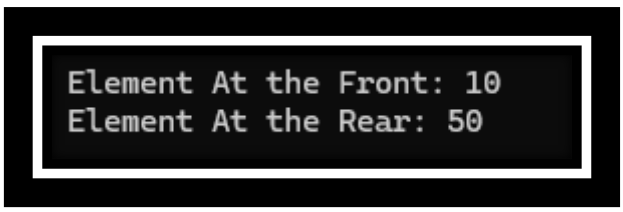
}//end of QList
```

Output:-

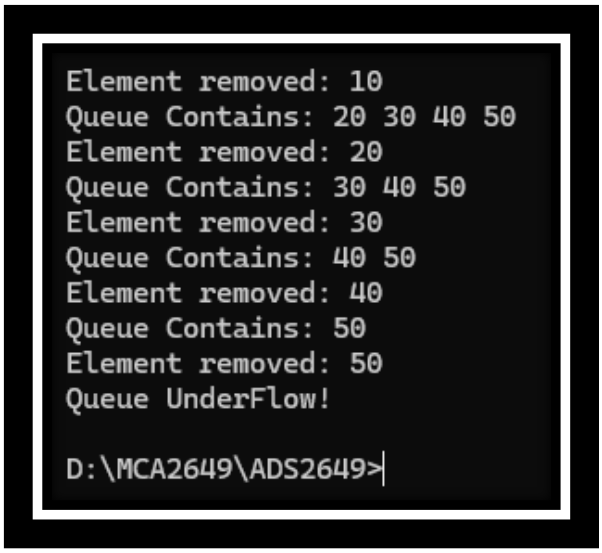
1)Enqueue



2)PeekFront & PeekRear



3)Dequeue



2) List Implementation of an Priority Queue

Source Code:-

```
//Name:Satish Rathod

//Roll No:MCA2649

//Module 4: Linked Lists

// Application of Linked Lists - List Implementation of an Priority Queue


import java.util.*;


//Priority Queue Node Templete

class PQNode

{

        int data;

        int pri;

        PQNode next;
```

```
        public PQNode(int d, int p)
        {
                this.data = d;
                this.pri = p;
                this.next = null;
        }

}//end of the PQNode
//Priority Queue templete
class PriQue
{
        PQNode front;
        PQNode rear;

        public PriQue()
        {
                front = null;
                rear = null;
        }

        //Enqueue
        public void Enqueue(int d, int p)
        {
```

```
//make a new node
PQNode x = new PQNode(d,p);


//First Node in the Priority Queue -Return Control
if(front == null)
{
        front = x;
        rear = x;
        return;
}


//Traqverse the Queue and fiind the correct place for x
PQNode prev = null;
PQNode temp = front;


while(temp!=null && temp.pri < p)
{
        prev = temp;
        temp = temp.next;
}
if(temp == front) //Front Node Insertion
{
        x.next = front;
        front = x;
```

```
                }
                else if(temp == null) //Rear Node Insertion
                {
                        rear.next = x;
                        rear = x;
                }
                else  //Any Other Node
                {
                        prev.next = x;
                        x.next = temp;
                }
        }//end of Enqueue
}//end of PriQue

class PQueue
{
        public static void main(String args[])
        {


        }//end of the main
}//end of the
```